# افهم التطبيق من الصفر

دليل بالعربي لغير المتخصصين • مشروع خريطة الموظفين

لو فتحت المشروع وحسيت إن الملفات كتير وإن كل حاجة ظهرت فجأة، فده طبيعي. الدليل ده بيبدأ من اللي بتشوفه بعينك، وبعدها يوصلك للكود بالتدريج. مش محتاج تكون عارف Angular أو الخرائط قبل ما تبدأ.

## إيه اللي هتفهمه؟

هتعرف التطبيق بيعمل إيه، وإزاي تشغله، والشاشة اتكونت من إيه، وبيانات الموظفين جت منين. وبعدها هنمشي وراء ضغطة واحدة ونشوف إزاي بتغير القائمة والخريطة مع بعض.

## طريقة قراءة مريحة

اقرأ الصفحات من ٢ إلى ٦ عشان تفهم الصورة العامة. بعد كده اقرأ شرح المخزن والفلاتر والتحليل. في الآخر هتلاقي قاموس بسيط وتمارين بإجاباتها وخريطة للملفات.

> مش مطلوب تحفظ أسماء الملفات أو تكتب الكود دلوقتي. هدفك الأول تعرف: مين بيحفظ البيانات؟ مين بيحسب؟ ومين بيعرض؟

## خريطة الدليل

٢ الفكرة • ٣ التشغيل • ٤ رحلة البداية • ٥ الشكل • ٦ البيانات • ٧ المخزن • ٨ قراءة الكود • ٩ الفلاتر • ١٠ الخريطة • ١١ اختيار موظف • ١٢ التغطية • ١٣ مثال عملي • ١٤ المصطلحات • ١٥ المراجعة • ١٦ الملفات.

# ١. التطبيق ده بيعمل إيه؟

ابدأ بالمشكلة اللي التطبيق بيحلها

تخيل إنك مسؤول في شركة ليها موظفين في أماكن مختلفة ومكاتب في القاهرة. عندك جدول بأسماء الموظفين وعناوين مواقعهم، لكن الجدول لوحده مش بيوضح توزيعهم على الأرض بسهولة.

التطبيق بيحول المواقع إلى نقط على خريطة. تقدر تشوف مين قريب من مكتب معين، وتبحث بالاسم، وتعرض قسم واحد، أو الموظفين التابعين لمكتب معين.

## مثال من الشاشة

الموظف له كارت في القائمة ونقطة على الخريطة. الكارت بيقول لك هو مين، والنقطة بتقول لك مكانه فين. الاتنين بيمثلوا نفس الشخص.

## شبه التطبيق بمكتب شغل

ملفات البيانات هي دولاب الملفات. المخزن هو السبورة اللي عليها البيانات والاختيارات الحالية. خدمة الخريطة هي الموظف اللي بيتعامل مع الرسم والحسابات. مكونات الشاشة هي واجهات العرض اللي بتشوفها.

> التطبيق الحالي نموذج تجريبي: فيه ٢٤ موظف و٣ مكاتب و٤ أقسام. البيانات محلية وخيالية، ومفيش قاعدة بيانات أو خادم خلفي متوصلين بيه حاليا.

## إيه اللي بييجي من الإنترنت؟

خريطة الأساس الرمادية وموارد تشغيل ArcGIS محتاجة اتصال بالإنترنت. أسماء الموظفين ومواقعهم موجودة في ملفات المشروع، وليست بيانات تتبع مباشر.

# ٢. تشغيل المشروع على جهازك

كل أمر له وظيفة بسيطة

## قبل التشغيل

لازم يكون Node.js و npm متثبتين. حسب تعليمات المشروع، النسخة المجربة من Node.js هي 24.15.0. افتح المشروع في المحرر، وبعدها افتح Terminal داخل فولدره.

```
node --version
npm --version
```

الأمرين دول بيعرضوا أرقام النسخ. لو الجهاز مش متعرف عليهم، راجع تثبيت Node.js ومسار تشغيله، واقفل وافتح المحرر بعد التثبيت.

## أول تشغيل

```
npm ci
npm start
```

الأمر الأول بينزل المكتبات بالإصدارات المسجلة في ملف القفل. استنى لحد ما يخلص، وبعدها شغل الأمر التاني. الأمر التاني بيبدأ سيرفر تطوير يعرض التطبيق على جهازك.

```
http://localhost:4200
```

افتح العنوان في المتصفح. سيب الـ Terminal شغال وأنت بتستخدم التطبيق. لو عدلت ملف وحفظته، سيرفر التطوير عادة بيحدث الصفحة. لإيقافه استخدم Ctrl+C داخل الـ Terminal.

التثبيت غالبا مرة في البداية أو لما الاعتماديات تتغير. التشغيل كل مرة بـ npm start. ولو فيه خطأ، اقرأ أول رسالة خطأ؛ ما تكررش الأوامر عشوائيا.

# ٣. إزاي الصفحة بتبدأ؟

من صفحة فاضية إلى شاشة كاملة




ملف index.html فيه علامة اسمها app-root. اعتبرها مكان فاضي مخصص للتطبيق. main.ts يطلب من Angular يبدأ ويضع المكون الرئيسي في المكان ده.

المكون الرئيسي App فيه router-outlet. ده مكان الصفحة المختارة. ملف app.routes.ts بيقول إن العنوان الرئيسي يفتح OperationsMapPage، وبيحمل كودها عند الحاجة.

ملحوظة مهمة في مشروعك: app.html موجود، لكنه مش القالب المستخدم حاليا. app.ts كاتب القالب داخله مباشرة. عشان تفهم الشاشة الفعلية افتح operations-map.page.html.

# ٤. مين عمل الشكل اللي شايفه؟

فرق بين الهيكل والمظهر والتصرف

### HTML: موجود إيه في الشاشة؟

زي هيكل شقة: هنا عنوان، هنا زر، هنا قائمة، وهنا مكان الخريطة. ملف operations-map.page.html بيجمع الهيكل العام، وكل جزء فرعي ليه قالب خاص به.

### SCSS: شكله عامل إزاي؟

زي الدهان وتوزيع المساحات: الألوان والخطوط والمسافات والخلفيات وشكل الكروت. معظم التصميم العام في styles.scss، وفيه تنسيق خاص بمكون الخريطة كمان.

### TypeScript: إيه اللي يحصل لما تستخدمها؟

زي تعليمات التشغيل: لما المستخدم يختار قسم نغير الفلاتر، ولما يضغط موظف نتحرك لمكانه، ولما يطلب تحليل نحسب المنطقة المحيطة بالمكتب.

```
<app-employee-filters />
<app-employee-list />
<app-coverage-tool />
<app-map-view />
```

دي أسماء مكونات المشروع. Component يعني جزء مستقل من الشاشة. الصفحة الكبيرة بتركب الأجزاء دي مع بعض بدل ما تحط كل الشغل في ملف واحد.

الأيقونات من UiIconComponent، وألوان الأقسام مشتركة بين القائمة والخريطة. أدوات الخريطة نفسها، زي التكبير والمفتاح، بتوفرها مكونات ArcGIS.

# ٥. البيانات وصلت منين؟

افهم الموظف كنصوص وموقع في نفس الوقت

في فولدر public/data ملف employees.geojson وملف offices.geojson. الأول للموظفين والتاني للمكاتب. ملف GeoJSON عبارة عن بيانات مكتوبة بصيغة منظمة ومعاها وصف للموقع الجغرافي.

## الموظف الواحد عبارة عن جزئين

الجزء الأول بيانات وصفية: رقم الموظف واسمه وقسمه ووظيفته وبريده ورقم المكتب التابع له. الجزء التاني موقع: نقطة محددة بإحداثيات.

```
fullName: "Ahmed Samir"
department: "Engineering"
officeId: 1
coordinates: [31.346235, 30.065346]
```

دي قيم حقيقية من السجل الأول في ملف العينة، مكتوبة هنا بشكل مختصر للتوضيح. officeId هو رقم رابط؛ نستخدمه للوصول لاسم المكتب من قائمة المكاتب.

## يعني إيه إحداثيات؟

تخيل الأرض عليها شبكة. خط الطول وخط العرض بيحددوا عنوان النقطة على الشبكة. ترتيب GeoJSON هو خط الطول ثم خط العرض. عكس الترتيب ممكن يحط النقطة في مكان غلط.

النظام المستخدم اسمه WGS 84 ورقمه 4326. الأرقام هنا درجات، مش كيلومترات. عشان كده حساب مسافة التغطية محتاج أداة جغرافية مناسبة.

شكل النقطة بييجي من geometry، وبيانات الكارت من properties. الكود كمان يستخدم حقول longitude وlatitude عند إنشاء النقاط للتحليل؛ لازم تفضل متوافقة مع موقع geometry لو عدلت البيانات.

# ٦. افهم EmployeeStore كأنه سبورة

ده الملف المفتوح عندك، وده دوره الحقيقي

المخزن بيحتفظ بحالة التطبيق المشتركة في الذاكرة أثناء التشغيل. يعني البيانات المحملة، والفلاتر الحالية، ونتيجة التغطية، ورقم الموظف المختار. هو مش قاعدة بيانات دائمة.

## كل خانة بتحتفظ بإيه؟

employees: كل الموظفين المحملين. offices: كل المكاتب. filters: البحث والقسم والمكتب المختار. coverage: آخر نتيجة تحليل أو لا توجد نتيجة. selectedId: رقم الموظف المختار أو لا يوجد اختيار.

## ليه أكتر من جزء يحتاج نفس المخزن؟

القائمة محتاجة الموظفين، والأرقام محتاجة عددهم، والخريطة محتاجة تعرف مين يظهر. لو كل جزء احتفظ بنسخة مختلفة، ممكن القائمة تقول حاجة والخريطة تقول حاجة تانية.

تخيل السبورة مكتوب عليها: القسم المختار = Engineering. كل جزء يحتاج المعلومة يبص على نفس السبورة. ده سبب وجود مخزن مشترك.

## مين بيملأ السبورة؟

المستخدم لما يغير فلتر، الصفحة بتطلب من المخزن يحدث الاختيار. Store. بيحمل طبقات البيانات، ويقرأ السجلات، وبعدها يحط الموظفين والمكاتب في الـ MapService

## هل الفلتر بيحذف موظفين؟

لأ. القائمة الأصلية بتفضل موجودة. بنحسب منها قائمة أصغر للعرض. ولما تمسح الفلتر، نقدر نرجع نعرض الباقي. تحديث الصفحة بيعيد التحميل؛ المخزن ده مش وسيلة حفظ للتعديلات.

# ٧. قراءة كود المخزن بدون خوف

اقرأ المعنى قبل الرموز

```
readonly employees = signal<Employee[]>([]);
```

اقرأ السطر كده: عندنا خانة اسمها employees، قيمتها قائمة موظفين، وبدايتها قائمة فاضية. Employee هو وصف شكل الموظف، وعلامة [] معناها قائمة.

## يعني إيه signal؟

متغير Angular بيتابع تغييره. لما يتحدث، الأجزاء المعتمدة عليه تقدر تتحدث تلقائيا. نقرأ قيمته بإضافة قوسين، ونغير القيمة بـ set أو update.

```
this.employees()
this.employees.set(newEmployees)
this.filters.update(current => ({ ...current, ...changes }))
```

الأول يقرأ القيمة الحالية. التاني يستبدل القائمة. التالت يبدأ من الفلاتر الحالية ويطبق عليها التغييرات الجديدة، مع الاحتفاظ بباقي الاختيارات.

## نتيجة محسوبة: computed

مش قائمة بنكتبها يدويا. هي نتيجة فلترة الموظفين حسب الاختيارات الحالية والتغطية. لو أي مدخل visibleEmployees اتغير، Angular يعيد حساب النتيجة عند الحاجة.

## كلمتين ممكن يلخبطوك

readonly نفسها ممكن تتحدث. و signal تاني؛ إنما قيمة الـ signal معناها هنا إن اسم الخانة ما نربطوش بكائن null معناها مفيش قيمة مختارة أو مفيش تحليل نشط، مش معناها صفر نتائج.

الخلاصة اللي تحتاجها هنا: signal تحفظ حالة، computed تحسب نتيجة من الحالة. أما effect في خدمة الخريطة فيطبق النتيجة على مكتبة الخريطة الخارجية.

# ٨. لما تختار قسم، إيه اللي بيتحرك؟

مثال كامل على انتقال الاختيار




```
(changed)="store.updateFilters($event)"
```

السطر في قالب الصفحة. changed هو اسم الحدث اللي المكون بيبعته، و$event هي البيانات اللي جاية مع الحدث؛ في المثال ده فيها اسم القسم المختار.

الفلاتر بتتجمع بشرط «و»: الاسم مناسب، والقسم مناسب، والمكتب مناسب، وداخل التغطية لو مفعلة. لو شرط واحد مش متحقق الموظف ما يظهرش.

البحث بالاسم بيتجاهل اختلاف الحروف الكبيرة والصغيرة ويمسح المسافات الزائدة من أول وآخر النص. لو مفيش نتائج، القائمة والخريطة يعرضوا صفر؛ مش كل الموظفين.

# ٩. الخريطة اتبنت إزاي؟

خريطة أساس وفوقها طبقات

تخيل خريطة مطبوعة، وفوقها ورق شفاف. ورقة عليها الموظفين، وورقة عليها المكاتب، وورقة بنرسم عليها منطقة التغطية. كل ورقة اسمها Layer أو طبقة.

## الخريطة الأساسية

في MapService بيتعمل كائن ArcGISMap بخريطة أساس gray. دي الخلفية اللي عليها الشوارع والمناطق. المكاتب والموظفين إضافات فوق الخلفية، مش جزء من صورتها.

## الطبقات الثلاث

officesLayer و employeesLayer من نوع GeoJSONLayer بيانات ملف؛ كل واحدة بتقرأ.

coverageLayer من نوع GraphicsLayer التحليل؛ بنرسم فيها شكل مؤقت ناتج عن.

## مين قرر ألوان النقط؟

ملف symbology.ts فيه Renderer، يعني قواعد رسم. للموظفين بيقرأ department: الهندسة أزرق، المبيعات برتقالي، الموارد البشرية أخضر، والتشغيل أحمر. المكتب معين أصفر أكبر من نقطة الموظف.

## مين يعرض الخريطة فعلا؟

بياخد كائن الخريطة من الخدمة. لما العرض يبقى جاهز يبعت arcgis-map فيه عنصر MapViewComponent حدث، والمكون ينادي initialize لتحميل البيانات وتجهيز العرض.

الفلترة بتتطبق على LayerView، يعني عرض الطبقة داخل الخريطة. الداتا الأصلية تفضل كاملة عشان تحليل جديد يقدر يراجع كل الموظفين.

# ١٠. فتح التطبيق واختيار موظف

إيه اللي بيحصل أثناء الانتظار والضغط؟

## أثناء التحميل

تظهر رسالة انتظار وأدوات البحث والتحليل تكون معطلة. الخدمة تنتظر تحميل طبقتي .loading بيبدأ بقيمة loadState الموظفين والمكاتب، وتقرأ بياناتهم وتضعها في المخزن.

بعدها تجهز تفاصيل الموظفين، وتجيب عرض طبقة الموظفين، وتحرك الخريطة بحيث تشمل البيانات. لو كل ده نجح، الحالة تبقى ready وتقدر تستخدم الأدوات.

## لو التحميل فشل

الحالة تبقى error وتظهر رسالة وإمكانية إعادة المحاولة. زر Retry بيعيد تحميل الصفحة عشان يبدأ تحميل الخريطة وطبقاتها من جديد.

## لما تضغط كارت موظف

مكون القائمة يرسل الموظف المختار للصفحة. الصفحة تستدعي selectEmployee في MapService. الخدمة تجيب النقطة باستخدام رقم الموظف، وتتحقق إنه لسه ضمن النتائج الظاهرة.

الخريطة تتحرك للنقطة وتقرب عليها، وتعمل تمييز بصري، وتحفظ selectedId، وتفتح Popup فيه تفاصيل الموظف. وصف محتوى التفاصيل موجود في popups.ts.

الحركة وفتح النافذة بيحصلوا بعد عمليات تستغرق وقت. الكود بيتحقق من حداثة الطلب عشان ضغطة قديمة ما ترجعش تغير اختيار أحدث. تغيير الفلاتر كمان بيمسح التمييز والتفاصيل القديمة.

# ١١. تحليل التغطية خطوة بخطوة

إزاي نعرف مين قريب من مكتب؟

اختار مكتب، واختار ٢ أو ٥ أو ١٠ كيلومتر، واضغط Analyze coverage. اختيار القيم لوحده ما ينفذش التحليل؛ لازم تبعت الطلب بالزر.

## الطلب والحساب

الخدمة تبحث .applyCoverage يرسل رقم المكتب والمسافة. الصفحة تمررهم إلى CoverageToolComponent عن المكتب في البيانات وتعمل نقطة بإحداثياته.

أداة geodesic buffer ترسم منطقة حول النقطة بالمسافة المحددة بالكيلومترات، مع حساب انحناء الأرض. الشكل تقريب هندسي لدائرة جغرافية، وليس دائرة محسوبة على صورة الشاشة.

## اختبار الموظفين

الكود يمر على كل الموظفين ويختبر هل نقطة الموظف داخل المنطقة أو على حدودها. يحفظ أرقام المطابقين في employeeIds، ويرسم نفس المنطقة في طبقة التغطية.

## ليه ممكن أشوف رقمين مختلفين؟

عدد الموجودين جغرافيا في المنطقة يشمل كل الأقسام والمكاتب. عدد الظاهرين بعد الفلترة يشمل بس اللي حققوا شروط البحث والقسم والمكتب كمان. ممكن موظف تابع لمكتب تاني يكون قريب جغرافيا من المكتب المختار.

> التغطية هنا قرب جغرافي، مش وقت سفر ولا مسافة قيادة ولا نطاق خدمة على الطرق. Clear coverage يشيل الشرط الجغرافي والرسم فقط، ويترك فلاتر البحث والقسم والمكتب زي ما هي.

# ١٢. جرب السيناريو ده بنفسك

تجربة بدون تعديل أي ملف

### الخطوة الأولى: افهم البداية

افتح التطبيق واستنى Map ready. شوف إجمالي الموظفين وإجمالي المكاتب. جرب تقارن عدد الموظفين الظاهرين بعدد كروت القائمة. إجمالي الفريق يفضل ثابت مع الفلترة؛ الاختيار الحالي هو اللي بيتغير.

### الخطوة الثانية: ابحث عن أحمد

اكتب Ahmed Samir في خانة البحث. المفروض يظهر الموظف المطابق. اضغط الكارت وشوف الخريطة بتتحرك والتفاصيل بتفتح. دي تجربة عملية لرحلة الاختيار من القائمة للخدمة.

### الخطوة الثالثة: اجمع شرطين

مع نفس البحث اختار Sales. الموظف أحمد في العينة تابع للهندسة، فالشروط مش هتتوافق والنتيجة تكون صفر. ده سلوك صحيح. ارجع All departments عشان يظهر تاني.

### الخطوة الرابعة: افصل القرب عن التبعية

اضغط Reset، وبعدها اختار مكتب ومسافة ونفذ التحليل. لاحظ المنطقة المظللة. أضف فلتر قسم وشوف عدد الظاهرين يتغير، بينما المنطقة المرسومة ثابتة.

### الخطوة الخامسة: امسح بطريقتين

جرب Clear coverage: الرسم يختفي لكن فلتر القسم يفضل. بعدين جرب Reset: يمسح كل الفلاتر والتغطية ويعيد مجال الخريطة عشان يعرض مواقع البيانات.

سؤال تراجعه أثناء التجربة: اللي اتغير هو البيانات الأصلية، ولا شروط العرض؟ الإجابة: شروط العرض. الموظفين ما اتمسحوش من الملفات.

# ١٣. قاموس الكود باللغة البسيطة

ارجع للصفحة دي لما تقابل كلمة جديدة

## Component / Service / Store

Component: مكان مشترك لوظيفة أو منطق محتاجه أكتر من جزء. Service: جزء من الشاشة له مدخلات وأحداث. Store: خدمة هنا دورها الاحتفاظ بالحالة المشتركة والنتائج المشتقة.

## input / output / inject

input: معلومة داخلة للمكون من الصفحة، زي قائمة الموظفين. output: حدث خارج منه، زي اختيار موظف. inject: طلب الخدمة من Angular بدل إنشاء نسخة يدويا في كل مكان.

## signal / computed / effect

signal: قيمة قابلة للتحديث ومتابعة التغيير. computed: نتيجة محسوبة من القيم. effect: تنفيذ عمل عند تغير القيم؛ هنا تحديث طريقة عرض النقط في ArcGIS.

## الرموز داخل HTML

```
{{ store.visibleEmployees().length }}
[employees]="store.visibleEmployees()"
(selected)="selectEmployee($event)"
```

الأقواس المزدوجة تعرض قيمة كنص. الأقواس المربعة تمرر قيمة لخاصية أو مدخل. الأقواس العادية تستمع لحدث وتنفذ التصرف المكتوب عند حدوثه.

## مصطلحات الخريطة

Extent: مستطيل يحيط بالمنطقة. Popup: نافذة التفاصيل. Renderer: قواعد رسم الرموز. Feature: موقع أو شكل ومعاه بيانات. async وawait: التعامل مع شغل بياخد وقت، زي تحميل ملف، مع انتظار نتيجته قبل الخطوة التابعة له.

# ١٤. اختبر فهمك بهدوء

أسئلة صغيرة وإجابات مباشرة

## لو عدلت لون Engineering، أروح فين؟

ملف department-colors.ts هو مصدر ألوان الأقسام المشترك. symbology.ts يستخدم اللون في رسم نقط الخريطة، والقائمة تستخدم نفس تعريف الألوان.

## لو عايز أفهم ليه موظف مش ظاهر؟

ابدأ بالفلاتر الحالية والتغطية. بعد كده اقرأ filterEmployees في employee-selection.ts؛ كل الشروط لازم تتحقق. جرب Reset قبل ما تفترض إن بياناته ناقصة.

## ليه المخزن في الأول فاضي؟

لأن التطبيق لسه بيبدأ. القوائم تتملأ بعد ما خدمة الخريطة تحمل الطبقات وتقرأ السجلات. القيمة الابتدائية الفاضية مش معناها إن الملف مفيهوش موظفين.

## إيه الفرق بين null وقائمة أرقام فاضية؟

فاضية معناها التحليل اتنفذ ولم يجد employeeIds معناها لا يوجد قيد تغطية. نتيجة تحليل فيها null تساوي coverage موظفين. الأولى تسمح بعرض الموظفين حسب باقي الفلاتر، والثانية تعطي صفر نتائج.

## ليه موجود providedIn: root؟

عشان Angular يوفر الخدمة على مستوى التطبيق. المكونات اللي بتطلب EmployeeStore هنا بتتعامل مع نفس النسخة المشتركة، فتشوف نفس الحالة.

لو عرفت تشرح لحد: «البيانات تتحمل، تتخزن، تتفلتر، وبعدها القائمة والخريطة يعرضوا نفس النتيجة» فأنت فهمت العمود الأساسي للمشروع.

# ١٥. افتح أنهي ملف بعد كده؟

خريطة قراءة مرتبطة بالمشروع الحالي

## ابدأ بالبيانات وشكلها

```
public/data/employees.geojson
public/data/offices.geojson
src/gis/models/employee.model.ts
src/gis/models/map-state.model.ts
```

الأول والثاني أمثلة البيانات. ملفات models بتوصف شكل البيانات والقيم المتاحة، زي المسافات المسموحة وحالة التحميل.

## افهم الحساب والحالة قبل تفاصيل الخريطة

```
src/gis/services/employee-selection.ts
src/gis/services/employee-store.service.ts
src/pages/operations-map/operations-map.page.html
```

اقرأ شروط الفلترة، وبعدها شوف المخزن بيستخدمها إزاي، وبعدها راقب الصفحة وهي بتمرر البيانات وتستقبل الأحداث من المكونات.

## بعدها ادخل على العرض الجغرافي

```
src/gis/services/map.services.ts
src/gis/config/symbology.ts
src/components/map-view/map-view.component.ts
```

اقرأ MapService وظيفة وظيفة: initialize ثم selectEmployee ثم applyCoverage. ما تحاولش تفهم الملف كله مرة واحدة. التفاصيل الإضافية موجودة في README.md و docs/CODE_WALKTHROUGH.md.

الدليل مبني على قراءة كود المشروع وملفات بياناته. الأمثلة التوضيحية لا تضيف خصائص جديدة للتطبيق. احتفظ بالدليل بجانب الكود، وطبق سيناريو صفحة ١٣ أثناء القراءة.